SleepWalk: Exploiting Context Switching and Residual Power for Physical Side-Channel Attacks

Sahan Sanjaya, *Student Member, IEEE,* Aruna Jayasena, *Member, IEEE,* and Prabhat Mishra, *Fellow, IEEE*

***Abstract*—Context switching is utilized by operating systems to change the execution context between application programs. It involves saving and restoring the states of multiple registers and performing a pipeline flush to remove any pre-fetched instructions, leading to a higher instantaneous power consumption compared to typical program execution. In this paper, we introduce a physical power side-channel leakage source that exploits the power spike observed during a context switch, triggered by the user-space `sleep()` interface. We observed that this power spike directly correlates with both the power consumption during context switching and the residual power consumption of the previously executed program. Notably, the persistence of residual power signatures from previous workloads extends the scope of this side-channel beyond extracting the register data during the context switch. Unlike traditional approaches that require analyzing full power traces, applying complex preprocessing, or relying on external synchronization triggers, this novel technique leverages only the amplitude of a single power spike, significantly simplifying the attack. We developed a power model to illustrate the feasibility of mounting end-to-end side-channel attacks using `sleep`-induced power spikes. Experimental results show that our framework can perform cryptographic key recovery for both AES and SIKE implementations on the Broadcom BCM2711 SoC.**

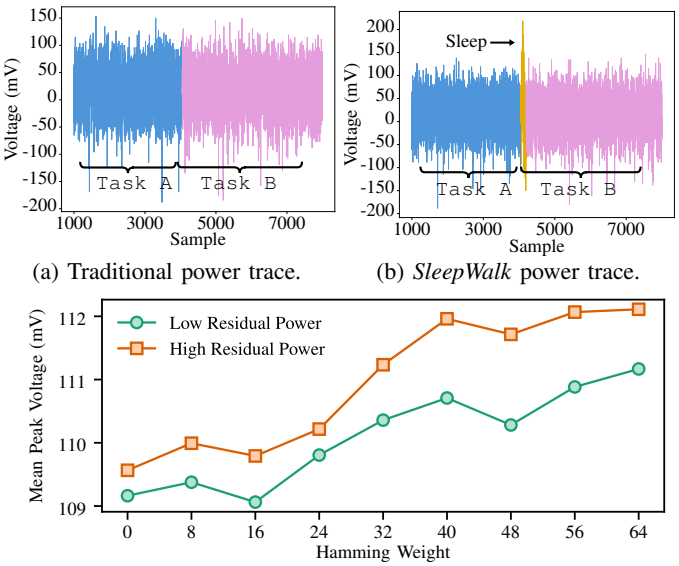
***Index Terms*—Power Side-channel Attack, Hardware Security, Context-switching, Residual Power, Cryptographic Key Recovery.**

I. Introduction

Context switching is utilized by operating systems to enable switching between tasks (application programs) [1], [2]. The basic idea behind context switching is to save (store) the state of a task, such as the values stored in registers and other resources, before switching to another task. When the original task is resumed, the previously saved values are restored (load), allowing the task to continue from where it left off. However, the concentrated load and store operations associated with context switching lead to higher power consumption due to frequent state transitions between '0' and '1' inside the chip, memory, and buses [3]. Consequently, this behavior creates a significant attack surface that can be exploited to extract secret information through physical power side-channel attacks [4].

A. Sleep-Induced Power Side-Channel

In this paper, we present a novel power side-channel (PSC) vulnerability, called *SleepWalk*, that leverages both context switching and residual power signatures to extract sensitive information. Although controlled context switches can be induced using various techniques [5], our approach primarily utilizes the user-space `sleep` interface, which inherently triggers a context switch. A distinctive power spike is observed at the onset of this transition, as illustrated in Figure 1b. Unlike traditional power side-channel analysis (as shown in Figure 1a), our approach relies solely on the amplitude of a single sampling point. This significantly reduces complexity by eliminating the need for high-end equipment, advanced preprocessing techniques such as trace alignment, or the extraction of high-dimensional features. Furthermore, the distinct and consistent nature of the observed power spike removes the need for external triggers to synchronize computation with trace collection. As illustrated in Figure 1c, this power spike is influenced by two key factors. The first is the **context-switching power signature** (depicted by the green line), which reflects the impact of the Hamming weight of register data at the moment the context switch occurs. The second is the **residual power signature**, representing the power footprint of instructions executed before the context switch. The difference between the green and orange lines in Figure 1c highlights the contribution of residual power, where the orange



(a) Traditional power trace. (b) *SleepWalk* power trace.

(c) Effect of the **context-switching power signature** (green line) and the **residual power signature** (represented by the power difference between the green and orange lines).

Fig. 1: We show the existence of a `sleep`-induced power spike (Figure 1b), which correlates with both the residual power consumption of the previously executed program and the power consumption during the context switch (Figure 1c). Unlike traditional power analysis (Figure 1a), this side-channel eliminates the need to analyze the entire power trace.

S. Sanjaya and P. Mishra are with the Department of Computer & Information Science & Engineering, University of Florida, USA. Email: {ssanjaya, prabhat}@ufl.edu. A. Jayasena is with the Department of Electrical and Computer Engineering, University of Massachusetts Amherst, USA. Email: arunaj@umass.edu

line, associated with high residual power, shows a greater amplitude than the green line, which corresponds to low residual power. Based on these observations, we introduce the *SleepWalk* side-channel vulnerability that exploits the `sleep`-induced power spike. Specifically, we demonstrate its significant potential as a security threat by extracting secret information from victim applications, as illustrated in Figure 2.

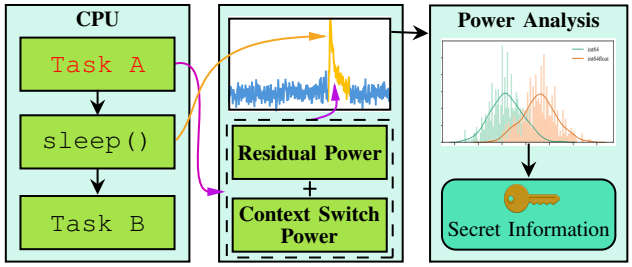


Fig. 2: Overview of *SleepWalk* power side-channel analysis, which extracts residual and context-switch power signatures from a single `sleep`-induced power spike to recover secret information from the victim application (`Task A`). Here, `Task B` denotes the next application scheduled after the processor yields from `Task A`. In the realistic attack setting, attacker-controlled code invokes `sleep()` immediately after the target application (`Task A`) returns. The resulting context switch produces a distinct power spike that contains side-channel leakage from the preceding application execution.

B. Threat Model

In this paper, we assume that the adversary has physical access to the device power supply or board-level voltage rails and can collect power traces during victim execution. The adversary can provide inputs to the target primitive, such as chosen plaintexts for AES or chosen ciphertexts for SIKE. The adversary can query publicly accessible AES encryption, decryption, and SIKE decapsulation interfaces and can repeat the target operation many times. We do not assume access to secret keys or privileged kernel code. The attacker also does not require control over the subsequently scheduled task.

We consider two adversarial capabilities. In *Adversary-1*, the adversary can modify the target application and insert a `sleep()` function call at a selected point. This model is used only for leakage characterization and proof-of-concept validation, since it allows us to isolate the context-switch and residual-power components of the *SleepWalk* channel. The controlled experiments in Sections III-B, III-C, IV, and V-A follow this model. In *Adversary-2*, the adversary cannot modify the target application's source code. Instead, the adversary executes a separate user-space program that invokes a public interface exposed by the target code through a linked library function and then calls `sleep()` immediately after the target function returns. The resulting context switch produces a sleep-induced power spike that contains leakage from the preceding target execution. Our realistic attacks in Sections V-B and VI are evaluated under this model.

C. Contributions

To demonstrate the potential of the *SleepWalk* side-channel, we explore a power model that enables practical side-channel attacks. Using this model, we showcase two attacks: key extraction from the post-quantum cryptographic algorithm SIKE, and key recovery from symmetric AES implementations. This paper makes the following key contributions:

- ***SleepWalk* Side-Channel Vulnerability**: We introduce *SleepWalk*, a novel power side-channel vulnerability that exploits a distinctive power spike generated during context switches invoked by the `sleep` function. This spike captures both context-switch and residual power signatures, revealing data-dependent behavior.
- **Residual Power Signature Analysis**: We develop a power model that characterizes the `sleep`-induced spike by isolating the contributions of context-switch power and residual power. We analyze how factors such as instruction count, data values, and register states affect the residual power signature.
- ***SleepWalk*-based Side-Channel Attacks**: Leveraging our power model, we demonstrate practical side-channel attacks, including cryptographic key recovery. *SleepWalk* enables a single-point leakage model, eliminating the need for trace alignment, external triggers, or complex preprocessing required in power analysis.
- **Key Recovery Attack on SIKE**: We perform a key recovery attack on the full secret key from the CIRCL SIKE implementation [6] using *SleepWalk*. We employ a probability density-based classification to successfully extract the full cryptographic key, showcasing the efficacy of this side-channel attack even against post-quantum cryptographic implementations.
- **Key Recovery Attack on AES**: We perform a key recovery attack on AES-128 [7], specifically targeting the final round secret key. We demonstrate that *SleepWalk* can be leveraged to identify processed data at byte-level granularity. Using this approach, we successfully recovered 10 out of 16 bytes of the final round key. Although this result does not outperform existing power side-channel attacks in terms of success rate, it shows that *SleepWalk* can reduce the key recovery complexity from 2^{128} to 2^{48} using only a single power trace point.

The remainder of this paper is organized as follows. Section II provides the necessary background and surveys related work. Section III introduces a new power side-channel. In Section IV, we develop a power model. Sections V and VI demonstrate key recovery attacks on SIKE and AES implementations, respectively. Section VII discusses possible mitigation strategies. Section VIII concludes the paper.

II. Background and Related Work

In this section, we first discuss the power models used in PSC attacks. Next, we describe the SIKE implementation and its key extraction attack. Finally, we survey related work.

A. Power Models

Power side-channel attacks rely on models that relate the measured power consumption of a device to the data processed during computation. Two widely used models are Hamming weight (HWT) [8] and Hamming distance (HD) [9]. The HWT

model estimates power consumption based on the number of '1's in a binary value, making it useful when power correlates with the number of set bits in registers or intermediate data. In contrast, the HD model captures transition-dependent power consumption by measuring the number of bit changes between two binary states. Beyond these standard models, application-specific attacks often require custom power models tailored to the target implementation and leakage source [10], [11].

B. Key Extraction from SIKE

Supersingular Isogeny Key Encapsulation (SIKE) is a post-quantum key encapsulation scheme based on supersingular elliptic curve isogenies [12]. It builds on SIDH and uses mechanisms such as the Fujisaki–Okamoto transformation to provide stronger security guarantees while maintaining compact keys and ciphertexts.

SIKE operations involve scalar multiplications, isogeny computations, and evaluations, which can expose zero-value intermediate states exploitable through side-channel attacks [13], [14]. By generating specific ciphertexts, an adversary can force zero coordinates in the Montgomery three-point ladder depending on consecutive secret key bits. Specifically, for a binary secret key k, when targeting bit $k[i]$ and assuming bits up to $k[i-1]$ are known, anomalous zeros occur during the $(i+1)$-th round of the Montgomery ladder if $k[i-1] \neq k[i]$. Once introduced, these zeros propagate through subsequent computations, enabling sequential recovery of the secret key.

C. Related Work

Software-based Power Trace Collection: Intel Running Average Power Limit (RAPL) has been utilized by many existing efforts to demonstrate power side-channel attacks [15]–[17]. DeepTheft [15] is an end-to-end attack that accurately reconstructs complex Deep Neural Network (DNN) architectures by exploiting the RAPL interface on x86 processors. PLATYPUS [16] is another software-based power side-channel attack that can successfully extract cryptographic keys. Zhang et al. demonstrated that unprivileged access to RAPL allows attackers to infer power consumption patterns, leading to leakage of sensitive information [17]. The Hertzbleed attack [13] transforms power side-channel vulnerabilities into remote timing attacks, allowing attackers to infer cryptographic keys.

Hardware-based External Power Monitoring: There are promising research efforts that utilize physical measurements to capture information about the power trace. Moos et al. [18] conducted a static power side-channel analysis on an ASIC implementation of a threshold-implementation prototype chip. They demonstrated that even hardware designs with provable first-order security are susceptible to information leakage through static power consumption, highlighting the necessity for enhanced countermeasures in cryptographic hardware. The Physical Layer Supply Voltage Coupling (PSVC) vulnerability was explored in [19]. This study revealed that the power side-channel signature of a physical processor core can propagate across design boundaries via the power rails. There are various efforts for power side-channel evaluations on pre-silicon hardware designs [3], [20]–[24]. For example, EvilCS [3] introduced a framework for assessing information leakage during context switches within security enclave kernels.

Limitations of Existing Efforts: Existing power side-channel attacks are conducted either by monitoring software interfaces or through direct physical measurements. These attacks typically rely on capturing the entire power trace [4], [9], [19], [25], [26] or several sampling points [11], [16], [27] for analysis. To enhance trace resolution, they employ advanced processing techniques and utilize external triggers for precise trace alignment. These requirements are expensive in terms of both time and cost, and therefore, such attacks may not be feasible in many application scenarios.

III. *SleepWalk* Power Leakage Channel

In this section, we demonstrate the existence of the `sleep()`-induced power spike that serves as the basis for the *SleepWalk* side-channel. We first describe the experimental setup and then characterize the spike under different context-switch mechanisms and workloads.

A. Experimental Setup

The experiments were conducted on a Raspberry Pi 4 model B single-board computer (SBC) with a BCM2711 System-on-Chip (SoC) [28] integrating an ARM Cortex-A72 CPU [29], which has been reported as a 28 nm device [30], [31]. We used the Debian GNU/Linux 12 (Bookworm) operating system [32] with Linux kernel 6.6 and glibc 2.36. For trace acquisition, we used a Keysight DSOX1102G oscilloscope [33]. The oscilloscope probe was connected to the 5 V and GND pins of the Raspberry Pi device. The oscilloscope was controlled using the Virtual Instrument Software Architecture (VISA) protocol to capture the traces. A simple running average with a window size of 10 was applied to smooth the traces before finding the peak values. We used Python scripts to automate the trace collection process and to facilitate peak value extraction. For each input, the peak value was captured multiple times and averaged to reduce the effect of measurement noise. The resulting averaged peak values were then stored for further analysis based on the requirements of each specific attack, including cryptographic key recovery attacks on SIKE (Section V) and AES (Section VI).

B. Sleep and Power Spike

To illustrate the process of creating the `sleep`-induced PSC, we use two programs, `Task A` and `Task B`, that execute different sequences of assembly instructions. We first describe the observations we made during the process of introducing a `sleep` function call from the operating system while these functions are executed. Next, we analyze the kernel and hardware mechanisms underlying these observations.

Observation: Figure 1 illustrates the power trace of the program executing two tasks: `Task A` and `Task B`. First, we collected the power trace of running the two tasks one after the other, as shown in Figure 1a. Next, we inserted the `sleep` function call between the two tasks under the *Adversary-1* model, as shown in Figure 1b. We observe that adding a `sleep` function between the two tasks generates a distinct

power spike in the captured trace. After reverse-engineering the `sleep` function's implementation, we identified that the power spike that we observed is tied to the power signature of a context switch. Next, we explain the reverse-engineered analysis supporting this claim.

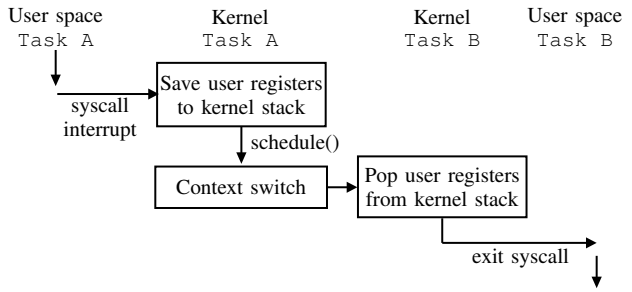


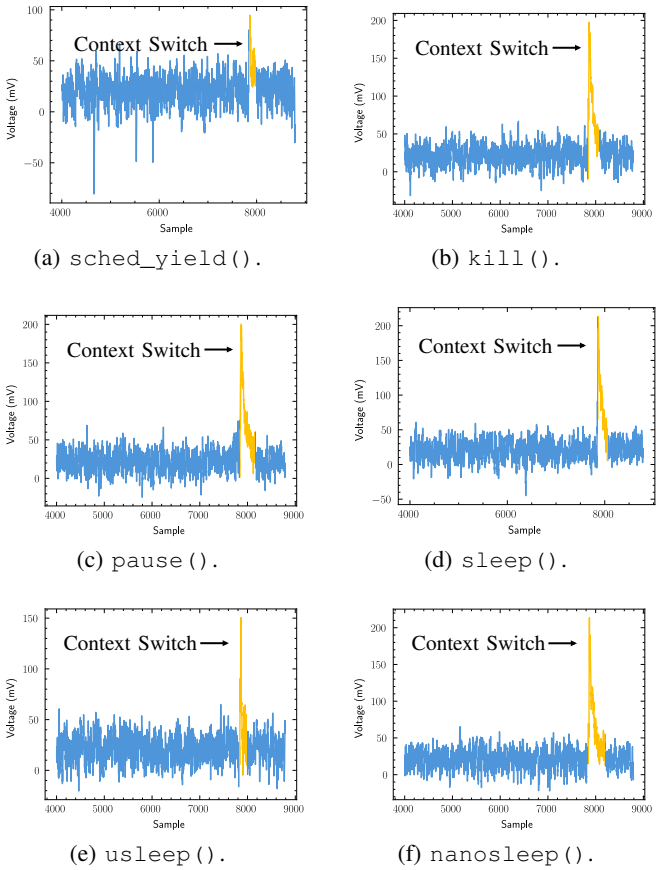
Fig. 3: Timing flow of Linux kernel context switch: saving context for `Task A` and restoring context for `Task B`.

Analysis of the Observation: Context switching is the process by which the kernel saves the state of the currently running process and restores the state of another process, allowing multiple processes to effectively share a single CPU. This mechanism is important for multitasking, and it is managed by the operating system kernel's scheduler. As shown in Figure 3, the `schedule` function is called when a context switch is required [2]. Since `schedule` is a kernel function, it cannot be invoked directly from user space. However, user-space applications can trigger context switches indirectly through various system calls or operations that cause the kernel to suspend the current process, prompting the scheduler to allocate CPU time to other processes. Figure 4 illustrates the function call trace during the execution of the `sleep` function [34]. It shows that invoking `sleep()` ultimately causes the kernel to invoke `schedule()`, triggering a context switch. Therefore, the power spike observed in Figure 1 is expected to reflect the power signature associated with a context switch. To confirm that the spike is indeed caused by the context switch, we conducted an experiment using various context switch invocation methods, including `sleep`, `usleep`, `nanosleep`, `sched_yield`, `pause`, and `kill` with the `SIGSTOP` signal [32].

```
sleep
 |->xnanosleep
    |->nanosleep
      |->clock_nanosleep
           |->invoke_syscall
              |->__arm64_sys_clock_nanosleep
                |->common_nsleep
                  |->hrtimer_nanosleep
                    |->do_nanosleep
                      |->schedule
```

Fig. 4: Call trace of the `sleep()` function.

As illustrated in Figure 5, the results confirm that the power spike is caused by the context switch. Additionally, `sched_yield` induces a less detectable power spike, while `kill` and `pause` require a process ID (PID), limiting their applicability in realistic attack scenarios. In contrast, `sleep`, `usleep`, and `nanosleep` are easier to use and generate more prominent power spikes, making them more suitable for our analysis. As a result, all experiments in this paper utilize context switches induced by the `sleep` function; hence, we name this leakage channel *SleepWalk*.



(a) `sched_yield()`. (b) `kill()`.

(c) `pause()`. (d) `sleep()`.

(e) `usleep()`. (f) `nanosleep()`.

Fig. 5: Power traces collected during the execution of user-accessible calls that trigger a context switch. All traces exhibit a distinctive power spike relative to normal execution.

C. Distinguishing Programs with SleepWalk

To show that the *SleepWalk* side channel can be leveraged for side-channel attacks, we performed an initial program-distinguishing experiment. For this experiment, under the *Adversary-1* model, we used the setup described in Section III-A and executed six different application programs from the Stress-NG [35] benchmark suite to evaluate how varying workloads affect `sleep`-induced power spikes. Each workload was executed for a fixed duration of one second before invoking the `sleep` function at the end. The resulting power traces were collected to compute the probability density of the peak power values. Figure 6 illustrates these distributions for each workload. While some overlap exists among the curves, the peak power values still provide distinguishability across workloads. Notably, the `int8`, `int32`, and `int64` workloads use the same function but operate on different integer sizes. The ability to differentiate between these three workloads suggests that *SleepWalk* can also be used to distinguish data types used by the previously executed programs.

The key takeaway from this experiment is that there is a correlation between the `sleep`-induced power spike and the executed program. Building on this insight, in Section IV, we

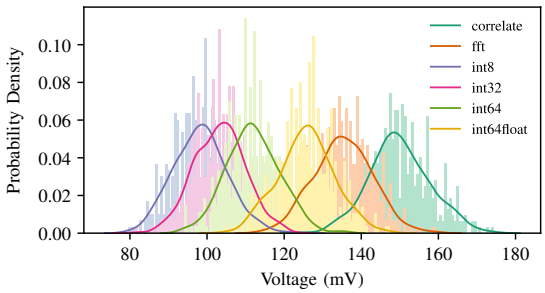


Fig. 6: Probability density of `sleep`-induced peak power values for six workloads. The `sleep` function was inserted at the end of each program, and each program was executed for one second.

develop a power model for the `sleep`-induced power spike to analyze its correlation with the data held in registers and processed by the CPU prior to the context switch.

IV. *SleepWalk* Leakage Power Model

PSC attacks exploit the relationship between measured power consumption and the operations or data processed internally. As discussed in Section II-A, power models approximate this relationship and help infer sensitive information from observed traces. Here, we use controlled HWT- and HD-based experiments to characterize the leakage behavior of the *SleepWalk* side channel.

A. Power Spike Components

As discussed in Section I-A, the `sleep`-induced power spike contains two main components: (1) the power consumption associated with the context switch, and (2) the residual power consumption from the previously executed program. While the context switch component directly corresponds to the power usage at the moment of switching and is relatively straightforward to interpret, understanding the residual component requires an in-depth analysis.

$$P_{\text{total}} = P_{\text{dyn}} + P_{\text{leak}} + P_{\text{sc}}. \tag{1}$$

$$P_{\text{leak}} \propto V_{dd} \cdot I_{\text{leak}} \quad , \quad I_{\text{leak}} \propto T^2 e^{-\frac{V_{th}}{nV_T}}, \tag{2}$$

In the context of our experimental setup, the total power dissipation in CMOS circuits, P_{total}, can be expressed as the sum of dynamic power, P_{dyn}, leakage power, P_{leak}, and short-circuit power, P_{sc}, as shown in Equation 1 [36]. Although dynamic power is typically the dominant component in digital CMOS logic, leakage power has become a significant source of power dissipation in modern digital integrated circuits due to increased transistor density [36]. This behavior is captured by Equation 2, where V_{th} is the threshold voltage, n is a process-dependent constant, and $V_T = kT/q$ is the thermal voltage [37]. According to Equation 2, leakage current (I_{leak}) is proportional to temperature (T) and thus leakage power (P_{leak}) increases with temperature (T) [37], [38].

$$C_{\text{th}} \frac{dT(t)}{dt} = P_{\text{total}} - \frac{T(t) - T_{\text{amb}}}{R_{\text{th}}}, \tag{3}$$

On-chip temperature change can be modeled using a simple lumped RC thermal network as illustrated in Equation 3, where C_{th} is the effective thermal capacitance, R_{th} is the thermal resistance to the ambient, and T_{amb} is the ambient temperature. Based on Equations 1, 2, and 3, we can construct the coupled relationship between power and temperature. Switching activity increases P_{dyn}, which in turn raises total power dissipation and chip temperature. The elevated temperature then further increases P_{leak}, creating a positive feedback loop that amplifies total power. This feedback loop typically converges to a stable power-thermal point [38].

Based on this power-thermal feedback loop, we identify two key factors that influence steady-state power behavior. The first is switching activity, which directly impacts P_{dyn} and is modeled using HWT and HD. The second is execution time, which affects temperature buildup; longer execution durations lead to higher temperatures and, consequently, increased leakage power. Execution time is quantified by the number of instructions executed. This interplay between power and thermal effects becomes observable in the `sleep`-induced power spike as the residual power signature.

To validate the preceding theoretical analysis of the observed power spike, we conducted four controlled experiments under the *Adversary-1* model, each designed to isolate a specific factor and examine its relationship to the two key components: context-switch power and residual power. These experiments were guided by the following four hypotheses.

- **Register-data HWT** affects the context-switch power component (Section IV-B).
- **Processed-data HWT** affects the residual power component (Section IV-C).
- **Processed-data HD** affects the residual power component (Section IV-D).
- **The number of executed instructions** affects the residual power component (Section IV-E).

Each data point in these experiments represents the average of 1,000 measured power spikes, collected using the experimental setup detailed in Section III-A.

B. Hamming Weight of Data in Registers

We start the experiment with the following hypothesis regarding the data in the registers during the context switch.

> Hypothesis 1: The Hamming weight of register data affects the `sleep`-induced power spike.

To analyze the impact of the number of '1's in the data stored in the registers before a context switch, we use the program illustrated in Figure 7. This program is designed to evaluate the effect of HWT on register values while keeping the other factors constant. Our execution setup consists of the following two phases:

- *Phase 1 [Constant Loop Execution]*: The first phase contains a loop that provides a reasonable gap between two `sleep`-induced power spikes. This phase remains unchanged across experiments with varying HWT values. The loop ensures sufficient execution time while maintaining a consistent baseline effect across all HWT experiments.

- *Phase 2 [Register Configuration]*: In the second phase, a value with a fixed HWT (`HWT_VALUE`) is loaded into the general-purpose registers to amplify the effect during the context switch. Next, a context switch is triggered using the system call to the `sleep` function.

```
const_loop:
    B const_loop
end const_loop:
    // Value with fixed HWT
    X0 = HWT_VALUE
    // Load all GPR with the same value
    MOV X1, X0
    ::
    MOV X26, X0
    // Make a system call for sleep
    ADR X0, timespec
    MOV X1, XZR
    MOV X8, #101
    SVC #0
```

Fig. 7: Code for context switch HWT experiment.

Through this controlled setup, we are able to analyze the relationship between HWT and the power variations observed during context switching, providing a clear understanding of HWT's impact on this side-channel. After collecting the traces, we isolated the peak amplitudes of the traces corresponding to the context switch events. Figure 8 illustrates the results obtained by varying the HWT. The data clearly demonstrate a positive correlation between HWT and mean peak power value, with the peak power increasing as the HWT increases. This behavior confirms Hypothesis 1.

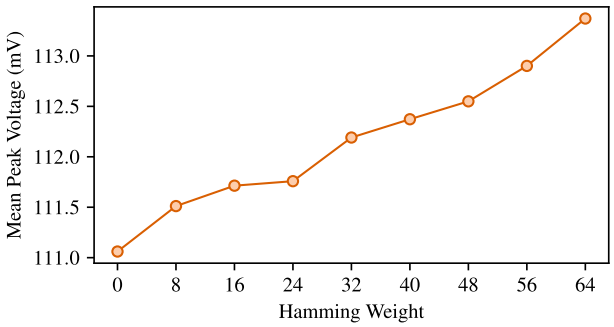


Fig. 8: Effect of varying the HWT of data in registers (`HWT_VALUE`) prior to the context switch. An increase in HWT results in an increase in the mean peak power.

C. Hamming Weight of the Processed Data

We designed this experiment to analyze the effect of the HWT of the processed data prior to a context switch. To evaluate this, we created the following hypothesis.

> Hypothesis 2: The Hamming weight of previously processed data affects the residual component of the `sleep`-induced power spike.

To observe the effect of the HWT of the data, it is essential to isolate it from the overall power signature. To achieve this, a fixed HWT value (`HWT_VALUE_LOOP`) is first loaded into the general-purpose registers. Next, we introduce an instruction loop that performs `OR` operations on the values stored in these

```
// Value with fixed HWT
X0 = HWT_VALUE_LOOP
// Number of loop iterations
X10 = LOOP_ITR
// Load all GPR with the same value
MOV X1, X0
::
MOV X26, X0
const_loop:
    // Perform OR operations
    OR X2, X1, X0
    ::
    OR X26, X1, X0
    B const_loop
end const_loop:
    // Value with fixed HWT
    X0 = HWT_VALUE
    // Load all GPR with the same value
    MOV X1, X0
    ::
    MOV X26, X0
    // Make a system call for sleep
    ADR X0, timespec
    MOV X1, XZR
    MOV X8, #101
    SVC #0
```

Fig. 9: Code for the residual power HWT experiment.

registers, as illustrated in Figure 9. Since all registers contain the same value, there are no bit transitions during execution, effectively eliminating any influence from the HD of the data. The `OR` operation is specifically selected because it preserves the HWT while eliminating the effect of HD. After executing this loop, another fixed HWT value (`HWT_VALUE`) is loaded into the general-purpose registers before invoking the `sleep` function. This setup creates two distinct HWT-related effects: one from the earlier `OR` operation loop, and another from the register values present at the time of the context switch. As discussed in Section IV-B, the HWT of data in the registers during a context switch affects the resulting power spike. If that were the only contributing factor, varying `HWT_VALUE_LOOP` while keeping `HWT_VALUE` constant should not change the mean power value.

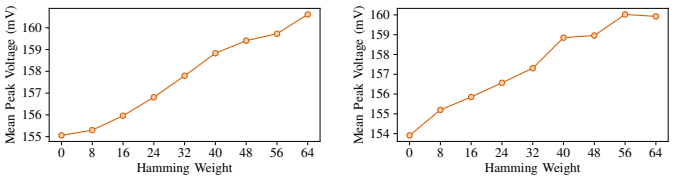
However, as shown in Figure 10, the HWT of data processed earlier also exhibits a strong correlation with the observed power signature, as expected due to the power-temperature interdependency. Thus, we validate Hypothesis 2, confirming that both the HWT of the data in the registers during the context switch and the HWT of the data processed prior to the context switch influence the observed power consumption.

D. Hamming Distance of the Processed Data

In order to evaluate the effect of bit transitions between `1` $\rightarrow$ `0` and `0` $\rightarrow$ `1` before a context switch, we focus on the HD of the processed data. We construct the following hypothesis to be evaluated during this experiment.

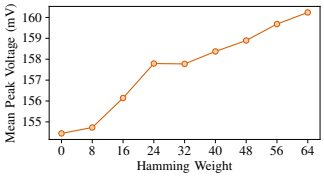
> Hypothesis 3: The Hamming distance of previously processed data affects the residual component of the `sleep`-induced power spike.

To evaluate the hypothesis, we created the program illustrated in Figure 11. The objective of this program is to isolate the effect of HD by controlling other variables,

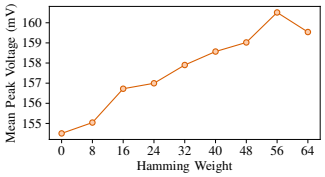


(a) HWT_VALUE = 0

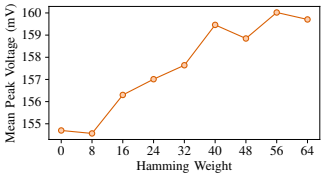
(b) HWT_VALUE = 16



(c) HWT_VALUE = 32



(d) HWT_VALUE = 48



(e) HWT_VALUE = 64

Fig. 10: Effect of the HWT of data processed prior to the context switch (HWT_VALUE_LOOP), reflecting residual power influence on the spike. The HWT of the data in the registers (HWT_VALUE) is held constant while varying the HWT of the data being processed inside the loop. Each figure illustrates the impact of HWT of data processed before the context switch when the register HWTs are fixed at 0, 16, 32, 48, and 64.

such as the HWT of individual states. To facilitate isolation, the program is designed with alternating groups of two LEFT SHIFT and RIGHT SHIFT operations. The reason for implementing SHIFT operations in pairs is based on the microarchitectural characteristics of the ARM Cortex-A72 processor, which features eight execution pipelines. Among these, two pipelines support integer ALU micro-operations, including logical shift-left (LSL) and logical shift-right (LSR) instructions [29]. The SHIFT operation can maintain the HWT of the input and output data within a certain range while introducing an HD effect. We implemented the shift operation using two input registers: one containing data with a fixed HWT, and the other containing the shift amount. The other general-purpose registers were also loaded with data of the same fixed HWT and were used as output registers for the SHIFT operations. Note that in each iteration, the same output registers are used for both LEFT SHIFT and RIGHT SHIFT operations. This design facilitates bit transitions within the registers beyond the ALU output, due to the alternating application of LSL and LSR instructions. We chose 0x00000fffffff00000 as the fixed value and defined SHIFT_VALUE as the shift amount. After the first iteration in the loop, when the shift operation is executed, the $2 \times$ SHIFT_VALUE bits immediately before the first '1' bit in the output register change from 0 to 1. At the same time, the last $2 \times$ SHIFT_VALUE '1' bits change from 1 to 0. This results in an HD of $4 \times$ SHIFT_VALUE in the output register and ALU output. For example, if SHIFT_VALUE $= 4$, after the first iteration in the loop, the output registers and ALU output contain 0x000000fffffff0000 (because of the RIGHT SHIFT), and when the LEFT SHIFT is performed (on 0x00000fffffff00000) the red-colored 8 bits transition from 0 to 1, while the green-colored 8 bits transition from 1 to 0, leading to a total HD of 16:

$$\texttt{0x000000ffffff0000} \rightarrow \texttt{0x0000ffffff000000}$$

The subsequent RIGHT SHIFT operation performs a similar transition (from 0x0000fffffff000000 to 0x000000fffffff0000), restoring the value in the output register to its initial state.

As illustrated in Figure 12, the HD of the data processed prior to the context switch influences the power spike amplitude observed during the context switch. This result supports Hypothesis 3 as well as the explanation of the power-temperature feedback.

```
// Read shift amount
X0 = SHIFT_VALUE
// Value with fixed HWT
X1 = 0x00000fffffff00000
// Number of loop iterations
X10 = LOOP_ITR
// Load all GPR with the same value
MOV X2, X1
::
MOV X26, X1
const_loop:
    // Perform Shift operations
    LSL X2, X1, X0
    LSL X3, X1, X0
    LSR X2, X1, X0
    LSR X3, X1, X0
    ::
    B const_loop
end const_loop:
    // Value with fixed HWT
    X0 = HWT_VALUE
    // Load all GPR with the same value
    MOV X1, X0
    ::
    MOV X26, X0
    // Make a system call for sleep
    ADR X0, timespec
    MOV X1, XZR
    MOV X8, #101
    SVC #0
```

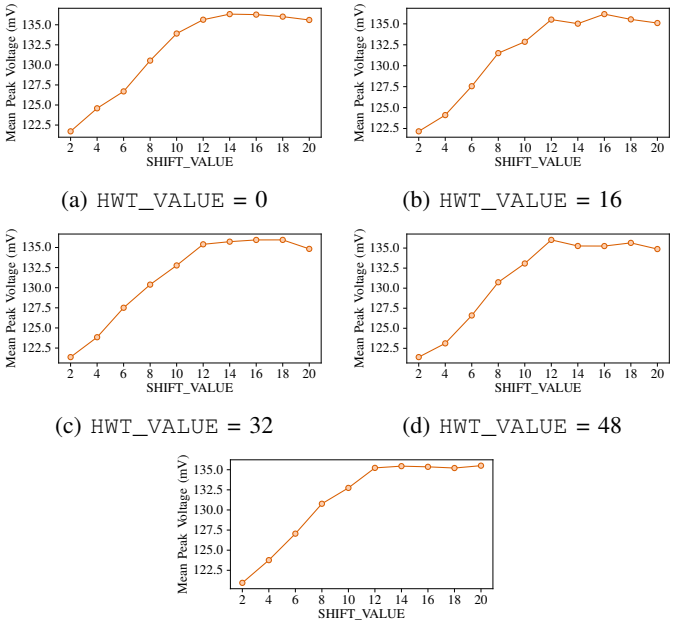
Fig. 11: Code for residual power HD experiment.

E. Number of Executed Instructions

To evaluate the impact of the number of instructions executed prior to a context switch on the sleep-induced power spike, we formulate the following hypothesis.

> Hypothesis 4: The number of instructions executed before the context switch affects the sleep-induced power spike.

To test this hypothesis, we conducted experiments using the setup described in Section III-A, systematically varying the number of instructions executed before the context switch. This was accomplished by adjusting the loop iteration count (LOOP_ITR) in the same programs used in our



(a) HWT_VALUE = 0 (b) HWT_VALUE = 16

(c) HWT_VALUE = 32 (d) HWT_VALUE = 48

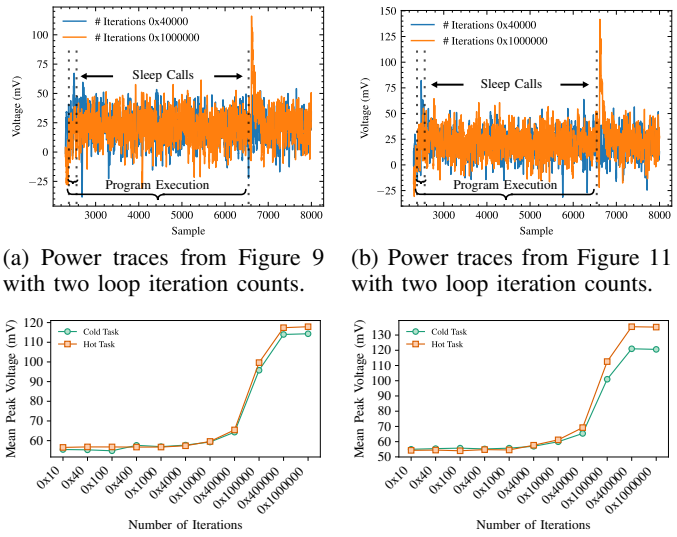
(e) HWT_VALUE = 64

Fig. 12: Effect of varying the HD of processed data prior to the context switch by changing SHIFT_VALUE, while keeping the HWT of the register data (HWT_VALUE) constant. Different HD values were obtained by varying the shift amount. Since 0x00000fffffff00000 is used as the fixed value, the resulting HD is $4 \times$ SHIFT_VALUE.

earlier experiments, as shown in Figures 9 and 11. In the experiment based on the program in Figure 9, we defined two scenarios: **Cold Task** and **Hot Task**. In the Cold Task, both HWT_VALUE_LOOP and HWT_VALUE were configured to produce a HWT of 0. In the Hot Task, both were set to a HWT of 64. In the experiment using the program from Figure 11, the Cold Task was configured with SHIFT_VALUE set to 2 and HWT_VALUE set to a HWT of 0, whereas the Hot Task used a SHIFT_VALUE of 20 and HWT_VALUE set to a HWT of 64.

The results of this experiment are presented in Figure 13. As shown in Figures 13a and 13b, a clear difference in spike amplitude emerges between programs with fewer loop iterations and those with more. This confirms that residual power is influenced by the number of executed instructions; specifically, an increase in instruction count amplifies the residual component of the sleep-induced power spike. Figures 13c and 13d further illustrate this effect by showing how varying the loop iteration count (LOOP_ITR) impacts both Cold and Hot Tasks for the programs in Figures 9 and 11. In both cases, the power spike amplitude increases with more iterations, and Hot Tasks consistently produce a higher mean peak power than Cold Tasks.

Figure 1c shows the combined effect of context-switch and residual components on the spike using the program in Figure 9. The low-residual setting uses a 0-HWT as HWT_VALUE_LOOP with LOOP_ITR = 0x100000, whereas the high-residual setting uses a 64-HWT as HWT_VALUE_LOOP with LOOP_ITR = 0x1000000.



(a) Power traces from Figure 9 with two loop iteration counts.

(b) Power traces from Figure 11 with two loop iteration counts.

(c) Effect of varying loop iterations in program 9.

(d) Effect of varying loop iterations in program 11.

Fig. 13: Impact of loop iteration count on the sleep-induced power spike. At low iteration counts, the mean peak power remains low. As the iteration count increases, the mean peak power rises, reflecting the influence of accumulated instruction execution on residual power.

F. Exploiting the SleepWalk Power Model

The presence of a residual power signature in the sleep-induced power spike significantly enhances the applicability and effectiveness of this novel side-channel. It enables the exploitation of both the data present in registers during the context switch via the power signature of the context-switch and information about previously executed programs and corresponding data via the residual power signature. We leverage this vulnerability to perform two distinct attacks, as demonstrated in the subsequent sections: a key recovery attack on SIKE (Section V) and a key recovery attack on AES (Section VI). Specifically, these two attacks explore two complementary strategies: a context switch power signature-dominant attack (Section V-A), where we insert or trigger context switches at precise points in the victim program to extract data in registers; and a residual power signature-dominant attack (Sections V-B and VI), where we repeatedly execute the same victim program to extract secret information based on accumulated power signatures.

V. Key Recovery Attack on SIKE

In this section, we perform a chosen-ciphertext proof-of-concept attack on SIKE using the setup described in Section III-A. Although SIKE has been deprecated due to unrelated security concerns, the purpose of this attack is to demonstrate how to exploit cryptographic algorithms with input-dependent intermediate computation using the sleep-induced power spike. For this experiment, we used Cloudflare's SIKE implementation in the Interoperable Reusable Cryptographic Library (CIRCL) [6].

As explained in Section II-B, anomalous zeros introduced during the Montgomery three-point ladder propagate to later

computations, including isogeny evaluations and the j-invariant computation. The introduction of zero-value states results in consistent patterns in power consumption, as arithmetic operations involving zero inputs are computationally simpler. This propagation, along with noticeable changes in the HWT within the registers, creates a strong and detectable signature in the power spike. Based on this input-dependent behavior in the SIKE algorithm, we conduct two separate experiments as follows:

- Proof-of-concept (PoC) attack on the SIKE implementation to evaluate feasibility, demonstrating a context-switch power-dominant attack.
- Attack on the original SIKE implementation in CIRCL, demonstrating a residual power-dominant attack.

A. Proof-of-concept attack

For this experiment, under the *Adversary-1* model, the adversary modified the CIRCL SIKE implementation by inserting `sleep()` immediately after the Montgomery three-point ladder function. This modification allowed us to determine whether anomalous zeros were present using the `sleep`-induced power spikes, characterizing this as a context switch power signature-dominant attack. Then, for a randomly selected key, we performed SIKE decapsulation with specially crafted ciphertexts. To evaluate the power signature difference for the conditions $k[i-1] \neq k[i]$ and $k[i-1] = k[i]$ for a target key bit i, we created two ciphertexts: one assuming $k[i-1] \neq k[i]$ and the other assuming $k[i-1] = k[i]$. For each key bit, we captured 1,000 traces and extracted the peak amplitude from each trace. Capturing 1,000 traces took approximately 5 minutes. Figure 14 illustrates the distribution of averaged peak amplitudes over the first ten bits of the secret key. The results confirm that there is a detectable difference in peak power amplitude between cases where an anomalous zero is triggered ($k[i-1] \neq k[i]$) and where it is not ($k[i-1] = k[i]$).

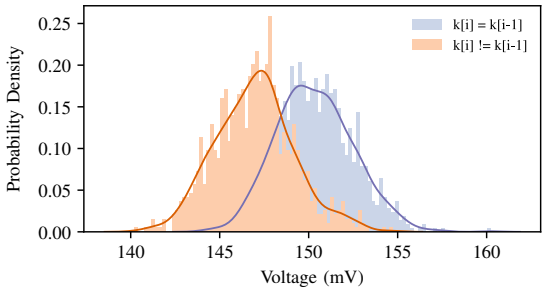


Fig. 14: Probability distribution of measured `sleep`-induced peak power amplitudes over the first ten bits of the secret key used in the SIKE implementation.

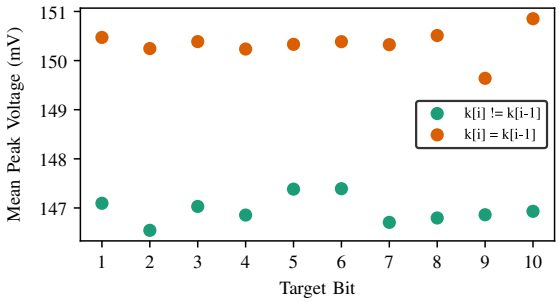
Next, we performed a complete key extraction attack with a threshold-based key bit identification approach. We plot the mean values of the measured peak amplitudes over the first ten bits of the secret key for both cases: when an anomalous zero is triggered ($k[i-1] \neq k[i]$) and when it is not ($k[i-1] = k[i]$). As shown in Figure 15a, there is a clear separation between the anomalous zero-triggered case ($k[i-1] \neq k[i]$) and the non-triggered case ($k[i-1] = k[i]$), indicating that a simple threshold classifier can distinguish the two cases. This separation occurs because we force a context switch immediately after the Montgomery three-point ladder operation; the registers are loaded with zeros in the anomalous zero-triggered case. Consequently, the `sleep`-induced power spike reflects a lower HWT as explained in Section IV. As shown in Figure 15a, when an anomalous zero is triggered, the mean value of the measured peak amplitudes is lower than in the non-triggered case.

B. SIKE Attack

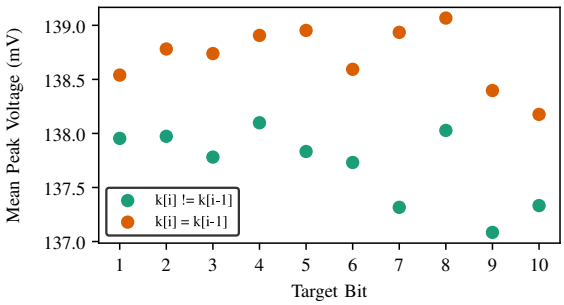
To create a realistic attack, we use the *Adversary-2* threat model. A separate user-space program invokes the SIKE decapsulation function and calls `sleep()` immediately after the function returns. This attack does not benefit from scenarios where context switching occurs immediately after anomalous zeros appear in the registers, as seen in the previous PoC SIKE attack. This limitation arises because the context switch is deliberately triggered only at the end of the decapsulation process. However, the measured peak values still contain residual power signatures from the executed program, and this residual power correlates with the HWT of the data processed during execution, as explained in Section IV. As a result, even when the context switch is applied at the end, the peak power values still carry information useful for key recovery.

During the experiment, we observed that a single decapsulation does not produce sufficient residual power for reliable key recovery. To enhance the amount of information captured in the peak values, we execute four iterations of decapsulation before forcing the context switch. With this amplification, we analyze the mean values of the measured peak amplitudes over the first ten bits of the secret key for both cases: when an anomalous zero is triggered ($k[i-1] \neq k[i]$) and vice versa ($k[i-1] = k[i]$). Figure 15b illustrates the results of this experiment. It can be observed that the results do not exhibit a clear global separation between the two cases ($= \& \neq$), unlike the previous *Adversary-1* SIKE attack shown in Figure 15a. For each bit position, the mean value for the ($k[i-1] \neq k[i]$) case is always lower than the mean value for the ($k[i-1] = k[i]$) case. This is caused by the residual power of the program retained within peak power values.

Unlike in the previous PoC SIKE attack, a threshold-based separation of two conditions is not feasible for this attack. Therefore, we employ Algorithm 1 to extract the key iteratively, one bit at a time. First, we set the target key bit $K[t]$ as the flipped value of the previously recovered key bit. That is, if $K[t-1]$ is 1, then $K[t]$ is initially set to 0, and vice versa. We then generate the specific ciphertext c using the function $generate_ciphertext$. Next, we execute the decapsulation setup with amplification using the function $perform_attack$ to collect the power traces and extract the peak values. The peak values are then identified using the function $find_peak$, and the mean peak value, $mean_value_1$, is computed using the function $mean$. If our guess for $K[t]$ is correct, the ciphertext c will induce anomalous zeros (since $K[t] \neq K[t-1]$) during SIKE decapsulation, resulting in a lower mean peak power value. Conversely, if our guess is incorrect, decapsulation will not produce anomalous zeros, as the ciphertext is generated using an incorrect key bit value, meaning $K[t] = K[t-1]$. This results in a higher



(a) Results of the *Adversary-1* SIKE PoC attack, where `sleep()` was invoked after the Montgomery three-point ladder.



(b) Results of the *Adversary-2* SIKE attack, where `sleep()` was invoked after the original SIKE decapsulation process.

Fig. 15: Mean values of the measured `sleep`-induced peak power amplitudes over the first ten bits of the SIKE secret key: when an anomalous zero is triggered ($k[i-1] \neq k[i]$; green data points) and vice versa ($k[i-1] = k[i]$; orange data points).

mean peak power value. Since we have only one measured mean peak power value, we repeat the procedure by setting $K[t] = K[t-1]$. If this is the correct guess, both cases will yield high mean peak power values in the same range. However, if this guess is incorrect, the first case (where $K[t]$ was flipped) will produce a lower mean value due to the presence of anomalous zeros, whereas the second case will yield a higher mean value. At the end of this process, these conditions are checked, and based on the results, the correct key bit $K[t]$ can be determined before proceeding to the next key bit. The threshold (`SEP_LIMIT`) is used to determine whether two mean values are significantly different or within the same range. The threshold is set by conducting multiple local experiments to find the minimum distance between cases where anomalous zeros are present and absent. By executing the attack described in Algorithm 1, we were able to recover the full secret key.

Algorithm 1 Key Recovery Algorithm

```
Recovered key: K ← 0
Target bit position: t
for t = 1 to SIKE_KEY_LENGTH do
    K[t] ← ¬K[t − 1]
    c ← generate_ciphertext(K, t)
    collected_traces ← perform_attack(c, K)
    peak_values ← find_peak(collected_traces)
    mean_value_1 ← mean(peak_values)

    K[t] ← K[t − 1]
    c ← generate_ciphertext(K, t)
    collected_traces ← perform_attack(c, K)
    peak_values ← find_peak(collected_traces)
    mean_value_2 ← mean(peak_values)

    if |mean_value_1 − mean_value_2| <SEP_LIMIT then
        K[t] ← K[t − 1]
    else if mean_value_2 − mean_value_1 ≥ SEP_LIMIT then
        K[t] ← ¬K[t − 1]
    end if
end for
```

VI. Key Recovery Attack on AES

In this section, we perform our final key recovery attack on an AES-128 software implementation using the *SleepWalk* side-channel. This attack assumes oracle access to both AES encryption and decryption. The only additional requirement in *SleepWalk* is that the attacker calls `sleep()` after encryption finishes, under the *Adversary-2* threat model, so that the resulting `sleep`-induced spike can serve as the leakage point. Unlike the SIKE implementation, which contains a set of anomalous zeros that can be generated using specific ciphertexts during the decapsulation process, no known attacks exploit this specific characteristic with the AES implementation. Additionally, standard AES-128 processes data at the byte level, restricting the possible HWT values in registers to a range of 0 to 8. We target the final round secret key, extracting one byte of the key at a time. For this attack, we use the experimental setup outlined in Section III-A.

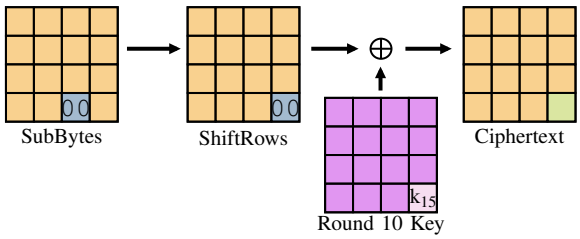


Fig. 16: Attack method for AES-128. For plaintexts generated using Algorithm 2 with a successful final round key byte guess (e.g., k_{15}), the corresponding bytes in the output of the SubBytes and ShiftRows are `0x00` (represented by blue-colored boxes). For all other incorrect key guesses, the values in the blue-colored boxes differ from `0x00`.

In an AES implementation, there are three main stages in the final round. As shown in Figure 16, these stages are:

1) The SubBytes (S-box) operation.
2) The ShiftRows operation.
3) The XOR operation with the final round secret key.

The final round is the ideal place to perform the attack since manipulating a value generated during the SubBytes (S-box) stage causes it to propagate to the ShiftRows stage. For example, when a byte becomes $0x00$ in the SubBytes stage, it propagates through the ShiftRows stage without modification, though its position may change. The corresponding byte in the final ciphertext will then be equal to the round 10 key byte at the same position as in the ShiftRows stage. Additionally, having the same value in both the SubBytes and ShiftRows

stages amplifies the effect of that value's HWT on the residual power, minimizing algorithmic noise. We perform three experiments under the *Adversary-2* model:

- **Level 1 AES Attack**: Reconstruct the SubBytes output.
- **Level 2 AES Attack**: Detect byte changes in the registers.
- **Level 3 AES Attack**: Extract the final round key.

Algorithm 2 Plaintext Generator Algorithm

1: **Inputs**: Target key byte position (T)
2: **Output**: Plaintext set (P)
3: Ciphertext: $C \leftarrow Rand()$
4: Key byte guess: k
5: **for** $k = 0$ to 255 **do**
6: $\quad C[T] \leftarrow k$
7: $\quad P[k] \leftarrow Decrypt(C)$
8: **end for**

A. Level 1 AES Attack

To perform a successful attack in real-world AES implementations, we have to address two key challenges: 1) how to manipulate a target byte in the final S-box stage and 2) how to distinguish single-byte HWT changes using `sleep`-induced power spikes. To solve the first challenge, we use Algorithm 2. It effectively generates plaintexts such that they produce different HWT values at the targeted byte location T in the final round ShiftRows stage of the AES implementation, for a given key byte guess of k. As shown in the algorithm, the attacker provides T as input. For example, suppose the target key byte position is 15 ($T = 15$). For each target byte, we start with a random 16-byte value (line 3). Next, we replace the target byte with the key guess (k) (line 6). Finally, the plaintext is generated by decrypting the corresponding ciphertext (line 7). This algorithm guarantees that when the key guess (k) is correct, the byte at location T in the final round ShiftRows stage becomes `0x00`, resulting in HWT 0. In all other cases, the byte at location T in the final round ShiftRows stage will have a value with an HWT greater than 0. Therefore, by identifying the HWT 0 case from the rest of the k guesses, we can recover the round 10 key byte. Following this approach, we derive three plaintexts such that four SubBytes outputs in the final round are either `0x00`, `0xFF`, or random, creating three scenarios where the four values have HWT 0, HWT 8, or a random HWT. The other 12 bytes remain unchanged across the three scenarios.

To strengthen the residual power effect, we executed 150,000 encryptions before invoking `sleep` to induce the power spike. This attack was conducted using the experimental setup described in Section III-A. For each plaintext, we captured 10,000 traces, totaling 30,000 traces per test set, and we conducted two tests. Capturing 10,000 traces took approximately 6 hours. We then measured the peak value of each trace and calculated the mean value for each scenario. Next, we calculated the mean difference of the observed peak values between the random HWT and HWT 0 (Random 1 – Four 0x00 Bytes), and between the random HWT and HWT 8 (Random 1 – Four 0xFF Bytes) in the first test set, as illustrated in the first row of Table I. We repeated the same calculation for the second test set as shown in the second row of Table I. By analyzing both mean difference values, we observe that the four `0x00` scenario exhibits the highest difference, making it the easiest to distinguish from four `0xFF` scenario. This confirms that the `sleep`-induced power spike can be used as a side-channel to recover the AES-128 final round key.

TABLE I: Comparison of average voltage differences for SubBytes outputs with random HWT, HWT 0, and HWT 8. The right two columns show voltage differences between random outputs and `0x00`, and between random outputs and `0xFF`. Rows correspond to the two test sets.

S-box Output	Four `0x00` Bytes	Four `0xFF` Bytes
Random 1	0.380544 mV	0.189312 mV
Random 2	0.345312 mV	0.113056 mV

B. Level 2 AES Attack

To evaluate the byte-level granularity of the *SleepWalk* side-channel, we performed the level 2 attack on the AES implementation. The goal of this attack is to evaluate the detectability of a single-byte change in the final round key as shown in Figure 16. Algorithm 2 with T = 15 was used to evaluate the ability to differentiate single-byte HWT changes using `sleep`-induced power spike values. We conducted the experiment using three different secret keys, collecting 5,000 traces, which took approximately 3 hours for each guessed HWT of the last key byte. We performed encryption using generated plaintexts, and to amplify the residual power effect, we executed 150,000 encryptions before invoking the `sleep` function to collect the peak power spike values.

Figure 17 presents the mean difference between the `sleep`-induced peak power of a given HWT and other HWT cases, evaluated over three different secret keys, with respect to varying HWT values in the last byte of the final round key. This means that the data point corresponding to HWT = 0 represents the mean difference in peak power values between the mean peak power of HWT = 0 and the mean peak powers of HWT = 1, 2, 3, 4, 5, 6, 7, and 8. We observe that HWT 0 exhibits the highest mean difference compared to other HWT values, making it detectable through its distinct peak power difference. This characteristic enables the classification of the HWT 0 case from the rest, allowing the key guess k corresponding to HWT 0 to directly reveal the actual final round key byte. Based on the results, we can conclude that it is possible to differentiate HWT 0 from the rest using `sleep`-induced power spike with single-byte resolution.

C. Level 3 AES Attack

Building on the Level 1 and Level 2 AES attack results, we developed the Level 3 AES attack to evaluate the effectiveness of extracting the full final round key. We conducted the same experiment as in Level 2, varying the target byte T in Algorithm 2 one at a time. For each target byte, we collected 5,000 traces over a period of approximately 3 hours and measured the mean peak power value.

Figure 18 presents the differences between the mean `sleep`-induced peak power of a given HWT and other HWT cases, with respect to varying HWT values at byte locations 15, 13, 3, and 1 of the final round key. We observe that for each

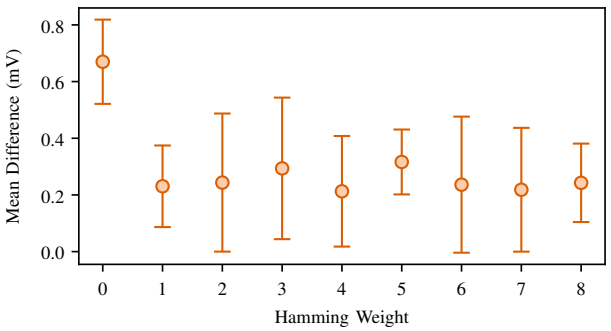
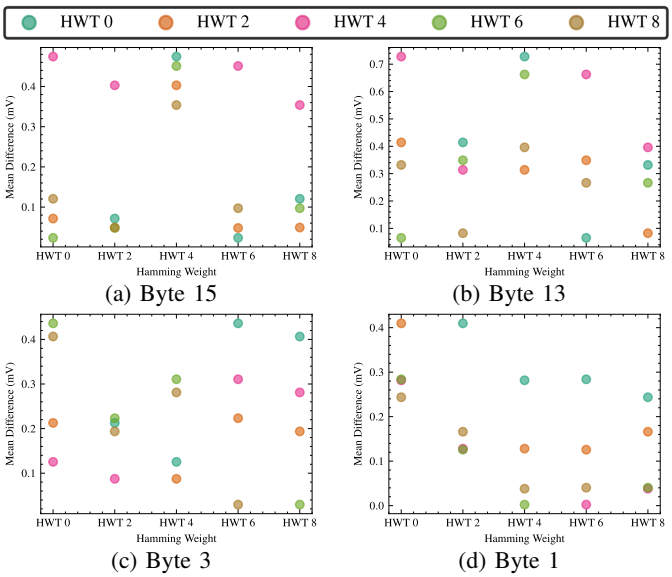


Fig. 17: Mean difference between the `sleep`-induced peak power of a given HWT and other HWT cases, over three different secret keys, with respect to varying HWT values in the last byte of the round 10 key.

target byte, the difference between HWT 0 and another HWT produces the highest peak power variations, making these two cases the extreme ends of the peak power distribution for a target byte across different HWT values. For example, in target byte 15, the highest peak power difference is observed between HWT 0 and HWT 4, while in target byte 3, it occurs between HWT 0 and HWT 6. Since HWT 0 consistently represents the lower end of the power distribution for each target byte, the attacker can identify the peak power corresponding to HWT 0 and, consequently, determine the correct key byte values. Because the AES attack relies on single-byte HWT variations, its leakage is weaker than the propagated zero-value leakage exploited in SIKE; consequently, we recovered only 10 out of 16 bytes of the final-round key using *SleepWalk*. For the six unrecovered bytes, the measurements did not yield stable candidate sets. We therefore conservatively retain all 256 candidates per byte, leaving a search space of $256^6 = 2^{48}$.



(a) Byte 15 (b) Byte 13

(c) Byte 3 (d) Byte 1

Fig. 18: Differences between the mean *sleep*-induced peak power of a given HWT and other HWT cases. Each colored data point shows the difference between the HWT indicated on the x-axis label and the HWT represented by the color.

VII. Mitigation of SleepWalk Vulnerability

Mitigating *SleepWalk* requires both detecting suspicious behavior and reducing leakage in the `sleep()`-induced power spike. Monitoring frequent `sleep()` calls or context switches may help identify repeated attacks, but is insufficient because `sleep()` is also used by benign applications and may be invoked only after the protected function returns. More effective defenses include clearing sensitive registers and intermediate values, executing fixed input-independent cleanup code before yielding the processor, and introducing scheduling noise to reduce the spike's signal-to-noise ratio.

Conventional side-channel countermeasures such as masking may also reduce data-dependent context-switch and residual-power leakage, although register reuse, compiler optimizations, or share recombination may still expose information. Our AES experiments use repeated independent encryptions to amplify residual leakage. Feedback-based modes, such as CBC, CFB, or OFB, may make this amplification more difficult because successive blocks depend on the prior state; however, the attack may remain feasible when the attacker can control or reset the initialization vector (IV), nonce, chaining state, or input sequence.

VIII. Conclusion

In this paper, we introduced *SleepWalk*, a novel power side-channel vulnerability that exploits the `sleep`-induced power spike to perform cryptographic key recovery attacks on off-the-shelf hardware. We demonstrated that the power spike generated when `sleep()` is invoked carries the signature of context switching power consumption and residual power consumption from the previously executed computations. We developed a power model for the `sleep`-induced spike and established its correlation with register data and data processed before the context switch. We demonstrated two attacks exploiting this vulnerability. First, we successfully extracted the full secret key from a constant-time SIKE implementation using a novel single-point side-channel, eliminating the need for trace alignment, external triggers, or advanced preprocessing techniques. Second, we developed a proof-of-concept key recovery attack on AES-128, showing that the `sleep`-induced power spike can be used to extract the final round key at byte granularity. Although we have evaluated the applicability of *SleepWalk* on an ARM Cortex-A72 Raspberry Pi, it is not inherently architecture-specific because it exploits data-dependent context-switch and residual-power behavior. A recent effort has observed similar leakage on RISC-V [39]. Future work will evaluate *SleepWalk* across architectures, OS scheduling, and power-management strategies.

Code Availability and Disclosure

The *SleepWalk* framework repository and case studies are available at github.com/Sanjaya97/SleepWalk. We disclosed our findings, including the PoC code, to Broadcom and ARM in January 2025. ARM indicated that the issue appears to originate at the SoC/board level and recommended coordinated disclosure with Broadcom. However, at the time of writing, Broadcom has not responded.

References

[1] C. Li, C. Ding, and K. Shen, "Quantifying the cost of context switch," in *Workshop on Experimental Computer Science*, 2007, pp. 1–4.

[2] Linux Kernel Labs, "SO2 lecture 3: Processes," 2023, accessed: 2025-03-21. [Online]. Available: https://linux-kernel-labs.github.io/refs/heads/master/so2/lec3-processes.html.
[3] A. Jayasena, R. Bachmann, and P. Mishra, "EvilCS: an evaluation of information leakage through context switching on security enclaves," in *Design Automation & Test in Europe (DATE)*, 2024, pp. 1–6.
[4] P. Kocher, J. Jaffe, and B. Jun, "Differential power analysis," in *International Cryptology Conference*. Springer, 1999, pp. 388–397.
[5] Y. Zhu, B. Chen, Z. N. Zhao, and C. W. Fletcher, "Controlled preemption: Amplifying side-channel attacks from userspace," in *ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025, pp. 162–177.
[6] A. Faz-Hernandez and K. Kwiatkowski, "Introducing CIRCL: An advanced cryptographic library," Cloudflare, 2019, accessed: 2025-01-10. [Online]. Available: https://github.com/cloudflare/circl.
[7] NIST, M. J. Dworkin, E. Barker, J. Nechvatal, J. Foti, L. E. Bassham, E. Roback, and J. F. Dray Jr., "Advanced Encryption Standard (AES)," 2001. [Online]. Available: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=901427
[8] T. S. Messerges, "Using second-order power analysis to attack DPA resistant software," in *International Workshop on Cryptographic Hardware and Embedded Systems*. Springer, 2000, pp. 238–251.
[9] E. Brier, C. Clavier, and F. Olivier, "Correlation power analysis with a leakage model," in *International Workshop on Cryptographic Hardware and Embedded Systems (CHES)*, 2004, pp. 16–29.
[10] Y. Xiang, Z. Chen, Z. Chen, Z. Fang, H. Hao, J. Chen, Y. Liu, Z. Wu, Q. Xuan, and X. Yang, "Open DNN box by power side-channel attack," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 67, no. 11, pp. 2717–2721, Nov. 2020.
[11] A. Kogler, J. Juffinger, L. Giner, L. Gerlach, M. Schwarzl, M. Schwarz, D. Gruss, and S. Mangard, "Collide+Power: Leaking inaccessible data with software-based power side channels," in *32nd USENIX Security Symposium*, 2023, pp. 7285–7302.
[12] M. Campagna, C. Costello, B. Hess, A. Jalali, B. Koziel, B. LaMacchia, P. Longa, M. Naehrig, J. Renes, D. Urbanik *et al.*, "Supersingular isogeny key encapsulation," *Submission to the NIST Post-Quantum Cryptography Standardization Project*, 2019.
[13] Y. Wang, R. Paccagnella, E. T. He, H. Shacham, C. W. Fletcher, and D. Kohlbrenner, "Hertzbleed: Turning power side-channel attacks into remote timing attacks on x86," in *31st USENIX Security Symposium*, 2022, pp. 679–697.
[14] L. De Feo, N. El Mrabet, A. Genêt, N. Kaludjerović, N. Linard de Guertechin, S. Pontié, and É. Tasso, "SIKE channels: Zero-value side-channel attacks on SIKE," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2022, no. 3, pp. 264–289, 2022.
[15] Y. Gao, H. Qiu, Z. Zhang, B. Wang, H. Ma, A. Abuadbba, M. Xue, A. Fu, and S. Nepal, "DeepTheft: Stealing DNN model architectures through power side channel," in *IEEE Symposium on Security and Privacy*, 2024, pp. 3311–3326.
[16] M. Lipp, A. Kogler, D. Oswald, M. Schwarz, C. Easdon, C. Canella, and D. Gruss, "Platypus: Software-based power side-channel attacks on x86," in *IEEE Symposium on Security and Privacy*, 2021, pp. 355–371.
[17] Z. Zhang, S. Liang, F. Yao, and X. Gao, "Red alert for power leakage: Exploiting Intel RAPL-induced side channels," in *ACM Asia Conference on Computer and Communications Security*, 2021, pp. 162–175.
[18] T. Moos, A. Moradi, and B. Richter, "Static power side-channel analysis of a threshold implementation prototype chip," in *Design Automation & Test in Europe (DATE)*, 2017, pp. 1324–1329.
[19] S. Sanjaya, A. Jayasena, and P. Mishra, "Information leakage through physical layer supply voltage coupling vulnerability," *IEEE Transactions on VLSI Systems*, vol. 33, no. 6, pp. 1715–1728, Jun. 2025.
[20] T. Zhang, J. Park, M. Tehranipoor, and F. Farahmandi, "PSC-TG: RTL power side-channel leakage assessment with test pattern generation," in *ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 709–714.
[21] N. Pundir, J. Park, F. Farahmandi, and M. Tehranipoor, "Power side-channel leakage assessment framework at register-transfer level," *IEEE Transactions on VLSI Systems*, vol. 30, no. 9, pp. 1207–1218, 2022.
[22] A. Jayasena, R. Bachmann, and P. Mishra, "CiseLeaks: Information leakage assessment of cryptographic instruction set extension prototypes," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 1551–1565, 2025.
[23] M. He, J. Park, A. Nahiyan, A. Vassilev, Y. Jin, and M. Tehranipoor, "RTL-PSC: Automated power side-channel leakage assessment at register-transfer level," in *IEEE 37th VLSI Test Symposium (VTS)*, 2019, pp. 1–6.
[24] A. Jayasena, E. Andrews, and P. Mishra, "TVLA*: Test vector leakage assessment on hardware implementations of asymmetric cryptography algorithms," *IEEE Transactions on VLSI Systems*, vol. 31, no. 9, pp. 1269–1279, 2023.
[25] A. Aysu, M. Orshansky, and M. Tiwari, "Binary Ring-LWE hardware with power side-channel countermeasures," in *Design, Automation & Test in Europe (DATE)*, 2018, pp. 1253–1258.
[26] N. Benhadjyoussef, M. Karmani, and M. Machhout, "Power-based side channel analysis and fault injection: Hacking techniques and combined countermeasure," *International Journal of Advanced Computer Science and Applications*, vol. 12, no. 5, 2021.
[27] M. Lipp, D. Gruss, and M. Schwarz, "AMD prefetch attacks through power and time," in *31st USENIX Security Symposium*, 2022, pp. 643–660.
[28] Broadcom Europe Ltd. and Raspberry Pi Ltd., *BCM2711 ARM Peripherals*, Raspberry Pi Ltd., 2022, accessed: 2026-05-07. [Online]. Available: https://datasheets.raspberrypi.com/bcm2711/bcm2711-peripherals.pdf.
[29] ARM Ltd., "Cortex-A72 software optimization guide," 2015, accessed: 2026-05-07. [Online]. Available: https://documentation-service.arm.com/static/5ed75eeeca06a95ce53f93c7.
[30] E. Upton, "Raspberry Pi 4 on sale now from $35," 2019, accessed: 2026-05-07. [Online]. Available: https://www.raspberrypi.com/news/raspberry-pi-4-on-sale-now-from-35/.
[31] ——, "Production and supply-chain update," 2022, accessed: 2026-05-07. [Online]. Available: https://www.raspberrypi.com/news/production-and-supply-chain-update/.
[32] M. Kerrisk, "Linux manual pages," accessed: 2026-05-07. [Online]. Available: https://man7.org/linux/man-pages.
[33] Keysight Technologies, *Keysight DSOX1102G User Manual*, 2023, accessed: 2026-05-07. [Online]. Available: https://www.keysight.com/th/en/assets/7018-05520/data-sheets/5992-1965.pdf.
[34] M. Kerrisk, "sleep(3) — Linux manual page," man7.org, 2025, accessed: 2026-05-07. [Online]. Available: https://man7.org/linux/man-pages/man3/sleep.3.html.
[35] C. I. King, "stress-ng," 2017, accessed: 2026-05-07. [Online]. Available: https://github.com/ColinIanKing/stress-ng.
[36] M. Pedram and S. Nazarian, "Thermal modeling, analysis, and management in VLSI circuits: Principles and methods," *Proceedings of the IEEE*, vol. 94, no. 8, pp. 1487–1501, 2006.
[37] Y. Liu, R. P. Dick, L. Shang, and H. Yang, "Accurate temperature-dependent integrated circuit leakage power estimation is easy," in *Design, Automation & Test in Europe*, 2007, pp. 1–6.
[38] A. Vassighi and M. Sachdev, *Thermal and power management of integrated circuits*. Springer Science & Business Media, 2006.
[39] S. Sanjaya and P. Mishra, "Sleep reveals the nonce: Breaking ECDSA using sleep-based power side-channel vulnerability," *arXiv preprint arXiv:2602.01491*, 2026.



Sahan Sanjaya is a fourth-year Ph.D. student in the Department of Computer and Information Science and Engineering at the University of Florida. He received his B.Sc. degree in Electronic and Telecommunication Engineering from the University of Moratuwa, Sri Lanka, in 2022. His research interests include side-channel attacks, hardware security, pre-silicon validation, and post-silicon validation.



Aruna Jayasena is a tenure-track Assistant Professor in the Department of Electrical and Computer Engineering at the University of Massachusetts Amherst. He received his Ph.D. from the University of Florida in 2025. His research focuses on secure computing architectures and systems security.



Prabhat Mishra is a Professor in the Department of Computer and Information Science and Engineering at the University of Florida. His research interests include embedded and cyber-physical systems, hardware security and trust, and energy-aware computing. He currently serves as an Associate Editor of ACM Transactions on Design Automation of Electronic Systems and ACM Transactions on Embedded Computing Systems. He is an IEEE Fellow, an AAAS Fellow, and an ACM Distinguished Scientist.